

Mem-W: Latent Memory-Native GUI Agents

 LV-NUS Lab, NUS &  NTU

 [GitHub](#)  [Mem-W-4B](#)  [Mem-W-8B](#)

Abstract

GUI agents are beginning to operate the web, mobile, and desktop as interactive worlds, where successful control depends on carrying forward visual, procedural, and task-level evidence beyond the fleeting present screen. Yet most agents still treat memory as an external, human-readable artifact: histories are summarized, categorized, retrieved, and reinserted as text or structured records before being encoded again by the policy. This creates a mismatch between the representational form in which experience is stored and the latent embedding sequence over which modern GUI policies actually act. We introduce Mem-W, a series of latent-memory-native GUI agents that treat memory as part of the agent’s continuous context rather than as an auxiliary symbolic scaffold. Mem-W weaves both historical trajectories (as experiential memory) and in-session segments (as working memory) into compact memory tokens through a shared trajectory-to-latent compressor. These tokens are woven with the current GUI observation and local context into one continuous embedding sequence, allowing the agent to read successes, failures, and unfinished progress through the same machine-native interface. Mem-W is trained with self-distillation and outcome-aware supervision to preserve decision-relevant state while filtering memory toward evidence that truly supports task success. Across four web and mobile navigation benchmarks, Mem-W consistently improves diverse backbones and memory-enhanced baselines, with gains of up to +30.0, suggesting that latent-context-native memory can serve as a scalable foundation for long-horizon GUI agency.

1 Introduction

GUI agents have begun to turn vision-language models (VLMs) into operators of web, mobile, and desktop interfaces [Nguyen et al. \(2025\)](#). Yet long-horizon GUI control is increasingly a context-interface problem, not only a perception or planning problem on a single screen. A task may require remembering constraints stated several screens ago, preserving partial progress after navigation ([Huang et al., 2025](#); [Zhou et al., 2026](#)), reusing a procedure observed in a similar application, or avoiding an action pattern that previously led to failure ([Cheng et al., 2026](#); [Zhu et al., 2026b](#)). The current observation is therefore insufficient, while the full raw history is too long, noisy, and visually redundant to be carried indefinitely. The central question is how past interaction evidence should be represented so that it becomes part of the agent’s operational context, rather than a detached record to be repeatedly translated back into model inputs.

Recent work has advanced agent memory along two main axes. One line maintains short-horizon state through explicit, human-readable structures over recent interaction, including graphs, lists, or layered records such as a *task-progress layer*, a *task-status layer*, and a *vision layer* ([Wang et al., 2025c](#); [Shi et al., 2026](#); [Li et al., 2026a,b](#); [Yuan et al., 2025](#)). Another line retrieves historical trajectories as long-horizon procedural, experiential memory, often organizing them into reusable abstractions such as a *tool-use experience layer*, *failure reminder layer*, or a *high-level planning layer* ([Wu et al., 2025b](#); [Fang et al., 2026](#)). These contributions have made memory increasingly effective,

yet they retain a deeper architectural commitment: before learning begins, the designer must decide what memory categories exist, how each is represented, and how each should interface with the policy. As a result, modern GUI memory designs juggle memory state trackers (Zhong et al., 2026; Cheng et al., 2026), summarizers (Gao et al., 2025; Ouyang et al., 2026; Wang et al., 2024), and retrieval indices (Liu et al., 2026, 2025a), each governed by its own format and update rule. This fixes the agent to a prescribed memory ontology that may not coincide with the structure demanded by downstream tasks (Zhang et al., 2025; Pan et al., 2026). The memory hierarchies that appear natural to the engineer need not be the ones most useful to the agent policy.

This observation suggests a more basic question: instead of designing separate mechanisms for each memory category, can a GUI agent become latent-context-native, storing and consuming experience in the same continuous space where its policy already operates? This work posits that **latent embedding space** offers a natural answer Zhu et al. (2025); Chen et al. (2025); Yu et al. (2026a). Because the agent already reasons over continuous embeddings, projecting trajectory evidence into the same space eliminates format mismatch between memory and agent policy, while letting end-to-end gradients (rather than hand-written rules) determine what is retained and how memories compose. Under this formulation, working memory and procedural memory differ only in *provenance* rather than *representation*: one is produced online from the current episode prefix, the other retrieved from an external bank of completed trajectories. Both enter the policy as tokens of the same type, and the structure of the resulting memory organization need not be prescribed but can emerge from training. To put the research question more formally:

Can a unified latent memory space, learned end-to-end, replace prescribed and heavily manual-engineered memory hierarchies for GUI agents?

To answer this question, we propose Mem-W, a series of latent-memory-native, or more broadly latent-context-native, GUI agents that weave dual-scale GUI evidence directly into the policy context. Mem-W learns a trajectory-to-latent compressor that maps variable-length observation-action segments into fixed-size latent blocks. At inference time, the same compressor receives both retrieved historical trajectories and the growing prefix of the ongoing episode, projecting each into token-efficient latent representations. The former freely becomes whatever **✿ experiential signal** the policy requires (e.g., procedures to imitate, failures to avoid); the latter distills **✿ working memory** state without exploding the context window. A shared latent space thus unifies **dual-scale GUI memory (✿ + ✿)** with no hand-engineered categorization or role-specific modules. The training of Mem-W is driven by two complementary signals: *self-distillation* from richer raw contexts and *outcome-aware supervision* that encodes procedural polarity between successful and failed evidence. In this way, memory is no longer an external scaffold imposed on the agent, but a latent context substrate through which experience, state, and action are jointly organized for GUI control. Our contributions can be summarized as follows:

- **Unified formulation.** We recast the increasingly intricate hierarchy of GUI memory as a single latent representation problem, arguing that short-horizon working memory and long-horizon experiential memory can be compactly grounded in one learnable space, where task-relevant structure emerges without prescribed taxonomies.
- **Learnable instantiation.** We instantiate this formulation with Mem-W, where a shared trajectory-to-latent compressor projects both retrieved historical trajectories and online episode prefixes into a unified latent space, trained through self-distillation and outcome-aware supervision.
- **Empirical evidence.** Experiments across four web and mobile navigation benchmarks show consistent gains of up to +30.0 over mainstream GUI agents and hand-engineered memory architectures, while further demonstrating favorable scaling behavior as the experiential memory bank grows with well-controlled inference delay.

2 Related Work

GUI Agent Memory. Drawing on existing memory taxonomies (Hu et al., 2026; Wu et al., 2025c), we group memory architectures for GUI agents into two broad families. **(I) Working memory** mitigates the token growth of long visual interaction histories within a single episode: some methods prune or crop historical screenshots at the visual-token level (Xu et al., 2026; Song et al., 2026), others compress history through KV-cache mechanisms (Huang et al., 2025; Zhou et al., 2026), while AndroTMem (Shi et al., 2026) preserves causally linked state anchors and MGA (Cheng et al., 2026) stores structured screenshot-spatial-memory tuples. **(II) Experiential memory** supports cross-session reuse of past experience, but appears in heterogeneous forms: AWM (Wang et al., 2024), ReasoningBank (Ouyang et al., 2026), ExpeL (Zhao et al., 2024), Memp (Fang et al., 2026), and EchoTrail-GUI (Li et al., 2026a) distill reusable workflows, lessons, or scripts; and HyMEM (Zhu et al., 2026a) combines symbolic strategies with continuous trajectory embeddings. Despite these advances, existing systems still prescribe, before learning, what memory types exist, how they are represented, and how they are filtered or composed—a design choice that has been shown not always to align with the memory structures demanded by downstream tasks (Zhang et al., 2025; Pan et al., 2026). Mem-W departs from this ontology-first paradigm by projecting all memory evidence into a shared learnable latent space, allowing task-relevant memory organization to emerge from end-to-end optimization.

Table 1: Comparison of representative memory mechanisms for GUI and multimodal agents. Working memory denotes in-session state retention, experiential memory denotes cross-session or retrieved task experience, latent memory denotes continuous memory tokens or embeddings, learnable indicates that the memory representation itself is trained rather than shaped only by predefined rules, and multimodal indicates support for GUI or other visual memory. ✓, ✗, and ● denote full, absent, and partial support.

Method	Working	Exper.	Latent	Native ctx.	Learnable	Multimodal	Agent	Memory interface
AWM (Wang et al., 2024)	✗	✓	✗	✗	✗	●	📄	Reusable textual workflows
ReasoningBank (Ouyang et al., 2026)	✗	✓	✗	✗	✗	✗	📄	Reasoning memories from self-evaluated experience
Memp (Fang et al., 2026)	✗	✓	✗	✗	✗	●	📄	Procedural instructions and script abstractions
EchoTrail-GUI (Li et al., 2026a)	✗	✓	✗	✗	✗	✓	📄	Reward-validated actionable trajectory memory
AndroTMem (Shi et al., 2026)	✓	✗	✗	✗	✗	✓	📄	Causally linked state anchors
MGA (Cheng et al., 2026)	✓	✗	✗	✗	✗	✓	📄	Dynamic structured state memory
HyMEM (Zhu et al., 2026a)	✓	✓	●	✗	●	✓	📄	Graph memory with trajectory embeddings
CoMEM (Wu et al., 2025b,a)	✗	✓	✓	✓	✓	✓	📄	Retrieved continuous trajectory embeddings
VisMem (Yu et al., 2026c)	✓	●	✓	✓	✓	✓	📄	Short-term and long-term latent vision memory
L ² -VMAS (Yu et al., 2026b)	●	✗	✓	✓	✓	✓	📄📄	Dual latent memories for visual multi-agent communication
Mem-W	✓	✓	✓	✓	✓	✓	📄	Unified latent context tokens

Latent memory. Latent space has emerged as a promising memory substrate (Hu et al., 2026; Yu et al., 2026a), owing to its token efficiency, machine-native format, and end-to-end learnability. For textual agents, a growing body of work encodes cross-session experience as latent embeddings, including NextMem (Zhang et al., 2026b), MemGen (Zhang et al., 2026a), SoftCoT/SoftCoT++ (Xu et al., 2025a,b), FlashMem (Hou et al., 2026) and others (Xu, 2026; Wang et al., 2025a,b). The idea extends naturally to multimodal agents: L2-VMAS (Yu et al., 2026b) and VisMem (Yu et al., 2026c) encode visual cues and trajectories through latent tokens, while CoMEM (Wu et al., 2025b,a) compresses historical trajectories into continuous embeddings to serve as experiential memory. Our work draws inspiration from this line, especially CoMEM (Wu et al., 2025b). We also benefit from the scaled GUI trajectory resources released by this work, which support part of our training pipeline as detailed later. However, Mem-W differs fundamentally from prior work: whereas existing methods typically employ latent representations for a single memory role, Mem-W uses a shared compressor and a unified latent space to subsume both working memory and experiential memory, allowing task-relevant memory structure to emerge through end-to-end training. Table 1 summarizes the comparison between Mem-W and contemporary memory-based agent methods.

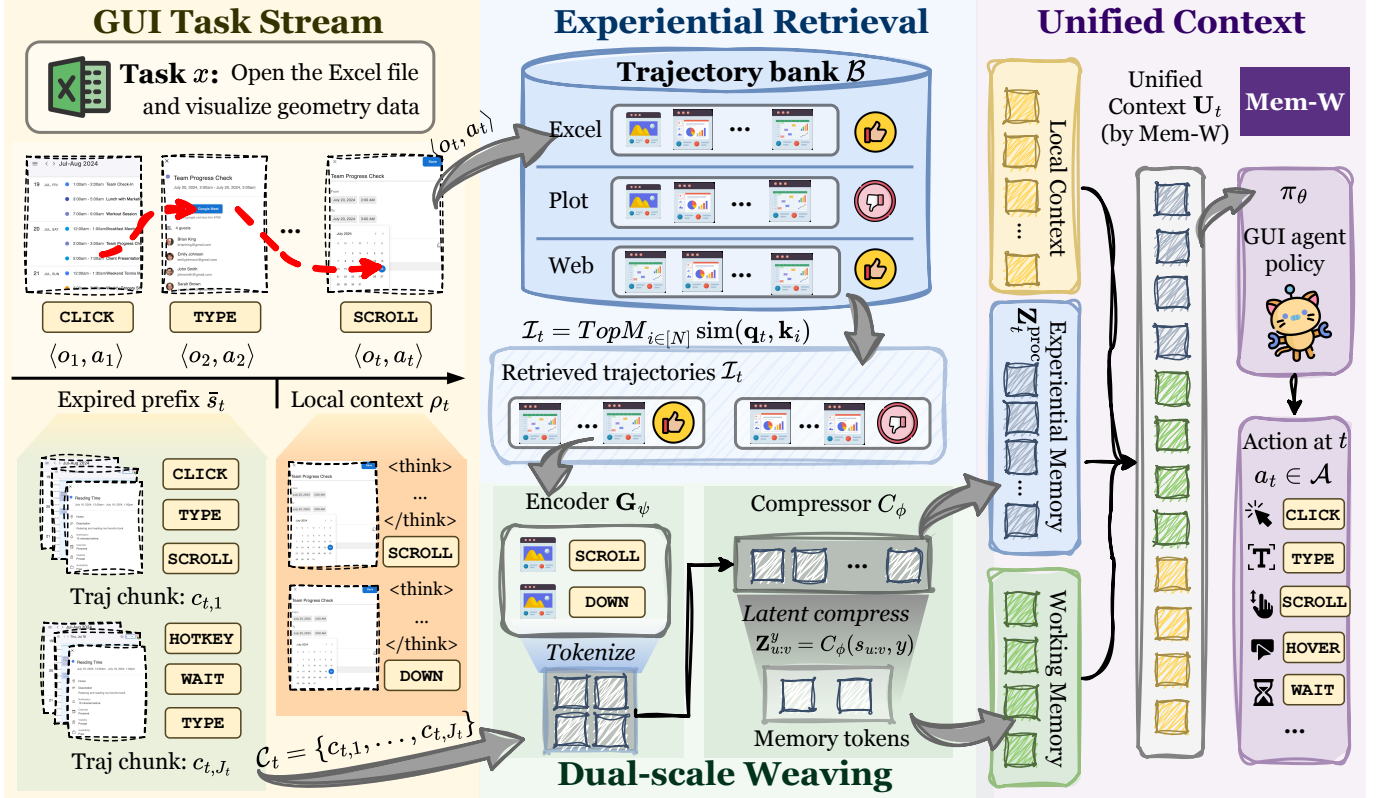


Figure 1: Overview of Mem-W. Given a GUI task stream, Mem-W retrieves relevant historical trajectories $\mathcal{B}$ as experiential memory and compresses expired in-session segments $\bar{s}_t$ as working memory through compressor C_ϕ . The resulting memory tokens are woven with the local context into a unified embedding sequence, enabling the GUI policy to act over both prior experience and current progress.

3 Method

As illustrated in Figure 1, Mem-W instantiates a latent-context-native interface for GUI agents. The goal is not to add another human-readable memory store beside the policy, but to convert interaction evidence itself into soft context tokens that live in the agent’s input embedding space. Short-horizon episode history and long-horizon retrieved experience are therefore exposed to the policy through the same continuous interface, allowing the agent to attend to them as native context rather than as externally serialized notes. This section formalizes necessary notations (▷ Section 3.1) and then presents the compressor (▷ Section 3.2), inference procedure (▷ Section 3.3), and training recipe (▷ Section 3.4).

3.1 Preliminaries and Overview

We consider an interactive GUI task specified by an instruction x . At step t , the agent receives an observation $o_t \in \mathcal{O}$, such as a screenshot, and predicts an executable action $a_t \in \mathcal{A}$. We write one interaction event as $e_t = (o_t, a_t)$ and a contiguous event sequence as $s_{u:v} = (e_u, \dots, e_v)$. A completed trajectory is denoted by $\tau = (x, s_{1:T}, y)$, where $y \in \mathcal{Y} = \{\text{succ}, \text{fail}\}$ is the terminal outcome label. The current episode is unlabeled before termination, while trajectories stored in an offline memory bank carry success or failure labels.

Let Π_θ denote the frozen GUI agent. We use $\mathbf{G}_\theta$ for its visual-textual encoder and $\mathbf{E}_\theta(x, o_t, \rho_t)$ for its ordinary input embeddings, where ρ_t is the bounded raw context retained around the current step. The goal of Mem-W is to equip this frozen agent with a compact latent memory interface, without updating the agent parameters θ . Formally:

$$\mathfrak{S} = \bigcup_{\ell \geq 1} (\mathcal{O} \times \mathcal{A})^\ell, \quad C_\phi : \mathfrak{S} \times (\mathcal{Y} \cup \{\emptyset\}) \rightarrow \mathbb{R}^{K \times d}, \quad (1)$$

where $\mathfrak{S}$ is the space of finite GUI event sequences, K is the per-segment latent-token budget, d is the embedding dimension of the GUI agent, and $\emptyset$ marks a segment whose final outcome is unknown. Action serialization $\alpha(\cdot)$ and outcome embedding $\gamma(\cdot)$ are implemented inside the compressor and are omitted from the abstract signature for notational compactness. The compressor is not a natural-language summarizer. It maps procedural GUI evidence into soft tokens that can be directly consumed by the frozen agent. At each decision step, Mem-W concatenates two sources of latent memory:

$$\mathcal{M}_t = [\mathcal{M}_t^{\text{proc}} ; \mathcal{M}_t^{\text{work}}], \quad (2)$$

where $\mathcal{M}_t^{\text{proc}}$ is distilled from retrieved historical trajectories and $\mathcal{M}_t^{\text{work}}$ is distilled from the expired prefix of the current episode. The two sources differ by provenance and role, not by representation: every individual $K \times d$ block in either memory is an output of the same compressor C_ϕ . Training therefore focuses on the compressor parameters ϕ , while keeping the GUI agent Π_θ frozen. The following subsections instantiate this interface with a query-based trajectory compressor, describe how procedural and working memories are woven into the agent context, and then give the two-stage training procedure.

3.2 Unified Trajectory-to-Latent Compressor

The compressor fixes the latent budget of each selected trajectory segment and places both memory scales in a shared continuous token space. Given $s_{u:v} = (e_u, \dots, e_v)$ with $e_i = (o_i, a_i)$, we pair each observation with a structured action serialization $\alpha(a_i)$ that records the action type and arguments. We also attach an outcome embedding $\gamma(y) \in \mathbb{R}^h$ for $y \in \mathcal{Y} \cup \{\emptyset\}$. The unknown marker $\emptyset$ is used for online working-memory segments whose terminal outcome is not yet available. The frozen encoder $\mathbf{G}_\theta$ converts the observation-action segment into contextual features:

$$\mathbf{H}_{u:v}^y = \mathbf{G}_\theta((o_u, \alpha(a_u)), \dots, (o_v, \alpha(a_v)); \gamma(y)) \in \mathbb{R}^{N_{u:v} \times h}, \quad (3)$$

where $N_{u:v}$ is the number of encoder features and h is the hidden dimension. The outcome embedding allows completed successful and failed trajectories to be encoded differently, while preserving the same interface for unlabeled online segments. We instantiate C_ϕ with a lightweight query transformer following the Q-Former design (Li et al., 2023). With K learned queries $\mathbf{Q} \in \mathbb{R}^{K \times h}$, the compressor extracts a fixed-size latent block and projects it into the agent embedding space:

$$\mathbf{Z}_{u:v}^y = C_\phi(s_{u:v}, y) = \mathbf{P}_\phi \left(\text{QFormer}_\phi \left(\mathbf{Q} + \mathbf{1}_K \gamma(y)^\top, \mathbf{H}_{u:v}^y \right) \right) \in \mathbb{R}^{K \times d}, \quad (4)$$

where $\mathbf{P}_\phi : \mathbb{R}^h \rightarrow \mathbb{R}^d$ is applied token-wise and $\mathbf{1}_K \in \mathbb{R}^{K \times 1}$ broadcasts the outcome embedding to all query slots. The output $\mathbf{Z}_{u:v}^y = [\mathbf{z}_1; \dots; \mathbf{z}_K]$ is never decoded into text; it is already a block of soft memory tokens for the agent.

3.3 Dual-Scale Memory Weaving at Inference

Given the compressor C_ϕ , inference consists of selecting a small number of relevant segments, compressing them into latent blocks, and placing those blocks before the ordinary agent context.

Procedural memory. The procedural memory is retrieved from an external trajectory bank $\mathcal{B} = \{(x_i, s_{1:T_i}^i, y_i)\}_{i=1}^N$. We reuse the frozen encoder $\mathbf{G}_\theta$ to form retrieval keys and the current query:

$$\mathbf{k}_i = \text{pool}(\mathbf{G}_\theta(x_i, s_{1:T_i}^i)), \quad \mathbf{q}_t = \text{pool}(\mathbf{G}_\theta(x, o_t, \rho_t)), \quad \mathcal{I}_t = \underset{i \in [N]}{\text{Top-}M \text{ sim}(\mathbf{q}_t, \mathbf{k}_i)}, \quad (5)$$

where M is the number of retrieved trajectories and $\text{pool}(\cdot)$ extracts the last hidden state. The retrieval encoder is frozen, and the discrete Top- M operation is not optimized by gradient descent. Let $(i_1, \dots, i_M)$ be the retrieved

indices sorted by descending similarity. The procedural memory is obtained by compressing the corresponding trajectories and concatenating them in retrieval-rank order:

$$\mathbf{Z}_t^{\text{proc}} = [C_\phi(s_{1:T_{i_1}}^{i_1}, y_{i_1}) ; \dots ; C_\phi(s_{1:T_{i_M}}^{i_M}, y_{i_M})] \in \mathbb{R}^{MK \times d}. \quad (6)$$

Successful and failed trajectories are therefore represented in the same latent space, with their outcome information encoded through $\gamma(y)$ rather than through hand-designed filtering rules.

Working memory. The working-memory path uses the same compressor. We keep the most recent L interaction steps in raw form,

$$\rho_t = s_{\max(1, t-L):t-1}, \quad (7)$$

and compress only the expired prefix. When $t > L + 1$, the expired prefix is $\bar{s}_t = s_{1:t-L-1}$; otherwise it is empty. We partition $\bar{s}_t$ into non-overlapping chunks of at most W steps, $\mathcal{C}_t = \{c_{t,1}, \dots, c_{t,J_t}\}$, where $J_t = \lceil |\bar{s}_t|/W \rceil$. If $\bar{s}_t$ is empty, then $J_t = 0$ and the working-memory block has zero rows. The chunks are compressed with the unknown-outcome marker and concatenated in temporal order:

$$\mathbf{Z}_t^{\text{work}} = [C_\phi(c_{t,1}, \emptyset) ; \dots ; C_\phi(c_{t,J_t}, \emptyset)] \in \mathbb{R}^{J_t K \times d}. \quad (8)$$

Thus, historical procedural evidence and the agent’s own past are compressed by the same mechanism, while remaining distinguishable by source.

Latent weaving. Let $\mathbf{E}_\theta(x, o_t, \rho_t) \in \mathbb{R}^{n_t \times d}$ be the ordinary input embeddings of the frozen GUI agent, including the current screenshot and the retained raw context. We add learned source embeddings $\mathbf{b}^{\text{proc}}, \mathbf{b}^{\text{work}} \in \mathbb{R}^d$ and construct the full latent-augmented input:

$$\mathbf{U}_t = [\mathbf{Z}_t^{\text{proc}} + \mathbf{1}_{MK}(\mathbf{b}^{\text{proc}})^\top ; \mathbf{Z}_t^{\text{work}} + \mathbf{1}_{J_t K}(\mathbf{b}^{\text{work}})^\top ; \mathbf{E}_\theta(x, o_t, \rho_t)], \quad (9)$$

where $\mathbf{1}_n \in \mathbb{R}^{n \times 1}$ broadcasts a source embedding to all rows of the corresponding block. The frozen agent then predicts the next action by

$$\pi_{\theta, \phi}(a_t | \mathbf{U}_t) = \Pi_\theta(a_t | \mathbf{U}_t). \quad (10)$$

Each selected segment contributes exactly K latent tokens, so the total memory context contains $(M + J_t)K$ latent tokens. In practice, we cap $M + J_t$ to a fixed constant, ensuring that the aggregate latent overhead remains bounded.

3.4 Training Latent-Memory GUI Agents

Since Π_θ is frozen, training only adjusts the compressor parameters ϕ . The objective is to make the latent blocks decision-relevant and usable by the frozen policy via two-stage training:

Stage 1: Compression fidelity via self-distillation. The first stage teaches the latent memory paths to replace raw historical context. We use the same frozen agent as both teacher and student. The teacher observes an extended raw window $\chi_t^{\text{raw}} = (x, s_{\max(1, t-L'):t-1}, o_t)$, $L' \gg L$, while the student observes the latent-augmented input $\mathbf{U}_t$ from Equation (9), where both working-memory blocks and retrieved procedural-memory blocks are constructed by C_ϕ . The self-distillation loss is:

$$\mathcal{L}_{\text{sd}} = \mathbb{E}_{(\tau, t)} [\ell_{\text{gui}}(\Pi_\theta(\cdot | \mathbf{U}_t), a_t^*) + \lambda D_{\text{KL}}(\text{sg}[\Pi_\theta(\cdot | \chi_t^{\text{raw}})] \parallel \Pi_\theta(\cdot | \mathbf{U}_t))], \quad (11)$$

where $\text{sg}[\cdot]$ stops gradients through the teacher path. Because θ is frozen, gradients from both the action loss and the KL term flow only into ϕ through the latent tokens in $\mathbf{U}_t$. The action-level loss anchors the compressor to the target behavior, while the KL term transfers the teacher’s extended-context distribution into the compressed representation, including both in-session working state and the retrieved experiential evidence.

Stage 2: Outcome-aware compressor optimization. The second stage also keeps both procedural and working memory active, and further optimizes their latent encoding with environment feedback. The frozen agent, conditioned on $\mathbf{U}_t$, rolls out full episodes in the GUI environment and receives a terminal reward $R(\tau) \in \{0, 1\}$. We optimize the compressor with a policy-gradient objective:

$$\min_{\phi} -\mathbb{E}_{\tau \sim \pi_{\theta, \phi}} \left[\sum_{t=1}^T \left(\log \Pi_{\theta}(a_t | \mathbf{U}_t) \cdot \hat{R}(\tau) - \beta D_{\text{KL}} \left(\Pi_{\theta}(\cdot | \mathbf{U}_t) \parallel \pi_{\text{ref}}(\cdot | \mathbf{U}_t^{\text{ref}}) \right) \right) \right], \quad (12)$$

where $\hat{R}(\tau) = R(\tau) - \bar{R}$ is a baselined reward, β controls the KL regularization strength, and π_{ref} is the frozen stage-one reference policy. The reference context $\mathbf{U}_t^{\text{ref}}$ is constructed with the frozen stage-one compressor, so the KL anchor remains fixed while C_{ϕ} is updated.

This stage does not backpropagate through the environment or the discrete retrieval operation. Instead, the terminal reward provides a policy-gradient signal on the sampled action log-probabilities, and gradients reach ϕ through the differentiable latent tokens inside $\mathbf{U}_t$. The compressor therefore learns not only what information to preserve, but also how to encode success- and failure-conditioned procedural evidence in a way that improves the frozen agent’s task performance. After training, inference uses the learned compressor C_{ϕ} , the frozen encoder $\mathbf{G}_{\theta}$ for retrieval, the trajectory bank $\mathcal{B}$, and the frozen GUI agent Π_{θ} . More details on the training process is detailed in Section A.

4 Experiments

4.1 Experiment Setup

Training data. We train Mem-W on web trajectories from CoMEM (Wu et al., 2025b) and the official mobile training set from (Lu et al., 2025). The web split contains 11,176 successful training trajectories across 13 domains, with a memory bank of 22,346 successful trajectories; the mobile split contains 2,489 successful training trajectories across six task categories, with a memory bank of 4,972 successful trajectories. Stage I trains C_{ϕ} for trajectory-to-latent compression via self-distillation, while Stage II further optimizes C_{ϕ} with outcome-aware supervision. Further data details are provided in Section B.1.

Evaluation Benchmark. Experiments are conducted on four challenging multimodal GUI benchmarks, including two web navigation benchmarks and two mobile navigation benchmarks:

- **Multimodal-Mind2Web** (Deng et al., 2023; Zheng et al., 2024) is a large-scale benchmark comprising over 2,000 open-ended tasks collected from 137 real-world websites across 31 domains. We directly adopt the test subset used in (Wu et al., 2025b). The evaluation metric is *success rate*, which measures whether the entire long-horizon task is completed successfully.
- **MMINA** (Tian et al., 2025) is designed to evaluate GUI agents on real-world websites. We use its shopping subset for evaluation, with *success rate* as the metric.
- **GUI-Odyssey** (Lu et al., 2025) evaluates cross-app GUI navigation on mobile devices, covering 6 mobile devices and 212 distinct applications. Its evaluation metrics include *Type Match (TM)*, which assesses whether the predicted action type matches the ground truth, and *Exact Match (EM)*, which further requires all action

Table 2: Performance comparison on AndroidControl-v2 (AC-v2) and GUI-Odyssey. For AC-v2-High/Low, we report Pass@1 and Pass@4, each decomposed into step accuracy (Acc), action type accuracy (Type), and grounding accuracy (Ground). For GUI-Odyssey, we report Type Match (TM) and Exact Match (EM). The baselines marked with † are reused from GUI-Libra.

Model	AC-v2-High						AC-v2-Low						Odyssey	
	Pass@1			Pass@4			Pass@1			Pass@4			TM	EM
	Acc	Type	Ground	Acc	Type	Ground	Acc	Type	Ground	Acc	Type	Ground		
Proprietary / Closed-source Models														
GPT-4o + UGround [†]	57.00	–	–	66.30	–	–	78.40	–	–	85.40	–	–	–	–
GPT-4.1 + UGround [†]	57.50	–	–	63.30	–	–	78.40	–	–	83.20	–	–	–	–
GPT-5-mini + UGround [†]	52.80	–	–	58.80	–	–	77.10	–	–	83.20	–	–	–	–
GPT-5 + UGround [†]	61.30	–	–	69.40	–	–	86.20	–	–	90.00	–	–	–	–
Open-source Models														
GUI-R1-3B [†]	40.00	–	–	54.00	–	–	55.80	–	–	71.90	–	–	–	–
GUI-R1-7B [†]	39.70	–	–	56.30	–	–	62.30	–	–	72.60	–	–	–	–
Aguvis-7B [†]	37.70	–	–	43.70	–	–	48.00	–	–	48.70	–	–	26.70	13.50
UI-TARS-1.5-7B	45.20	70.60	57.40	63.10	84.17	72.65	48.50	71.36	80.27	70.90	89.95	87.44	72.13	50.98
GLM-4.1V-9B-Thinking	37.20	67.84	48.43	49.00	74.87	59.64	67.10	90.20	67.26	73.40	93.22	78.48	67.26	32.88
GUI-Owl-1.5-8B	40.45	59.05	60.09	52.76	69.60	73.09	77.14	86.68	88.34	84.42	93.47	92.38	79.24	58.95
Qwen3-VL-2B	45.48	64.07	59.64	59.80	73.62	73.99	77.64	85.18	87.00	84.92	89.95	93.72	58.85	38.70
Qwen3-VL-8B	54.80	69.35	67.71	66.10	77.89	79.82	77.60	82.91	90.27	83.20	86.93	94.17	74.96	48.73
Qwen3-VL-32B	59.05	74.62	69.96	70.10	82.16	77.58	84.42	89.95	91.83	88.69	92.21	94.17	80.62	56.69
MAI-UI-2B	42.71	61.06	66.37	43.47	65.33	62.78	49.25	64.32	82.51	54.77	68.09	87.00	70.12	47.95
Step-GUI-4B	59.30	74.37	69.96	71.36	83.17	80.72	68.84	75.13	88.79	82.16	86.68	94.62	64.99	44.94
GUI-Libra-7B [†]	59.30	–	–	67.30	–	–	85.20	–	–	90.70	–	–	–	–
GUI-Libra-8B [†]	64.30	–	–	70.60	–	–	88.90	–	–	91.70	–	–	–	–
UI-Venus-1.5-30B-A3B	51.26	67.84	64.13	64.07	74.62	75.34	79.65	87.94	87.44	87.44	91.21	91.93	80.56	60.64
Mem-W Models														
Qwen3-VL-4B	49.30	63.57	69.06	63.30	75.88	77.13	78.90	85.93	90.58	82.40	88.69	93.72	72.89	44.73
↪ Mem-W-4B (Ours)	63.07	74.62	70.85	74.87	83.92	80.72	84.92	90.95	92.58	89.20	93.72	93.72	75.07	46.93
UI-Venus-1.5-8B	61.06	74.62	61.06	70.10	80.40	82.51	81.66	88.44	91.03	87.19	93.22	94.17	83.12	62.45
↪ Mem-W-8B (Ours)	68.59	81.16	71.30	80.40	88.94	82.96	87.94	93.72	89.24	94.22	98.24	94.62	84.88	65.05

parameters to be predicted correctly.

- **AndroidControl-v2** (Li et al., 2024; Yang et al., 2026), introduced by GUI-Libra (Yang et al., 2026), revises AndroidControl (Li et al., 2024) by correcting label errors and translating non-natural symbolic action histories. It adopts three major metrics, including step-wise accuracy, grounding accuracy and action type accuracy.

Mem-W Details. We apply the training procedure described in Section 3.4 to three mainstream GUI backbones: UI-TARS-1.5-7B (Seed, 2025), Qwen3-VL-4B (Bai et al., 2025), and UI-Venus-1.5-8B (Gao et al., 2026). Notably, the backbone parameters θ remain frozen throughout; only the compressor parameters ϕ are trained. At evaluation time, all models operate under a ReAct-style paradigm, outputting structured reasoning followed by a tool invocation from a predefined GUI action set (see Section B.3). The latent token count K is set as 8, the working memory window L equals 3, chunk size W is 4. The number of trajectories retrieved (M) in Equation (5) is 5. More parameters are detailed in Sections B.2 and B.3.

Baselines. We compare Mem-W with: (I) **open-source GUI agents**, including UI-TARS-1.5-7B (Seed, 2025), Qwen3-VL-2/4/8/32B (Bai et al., 2025), UI-Venus-1.5-8B/30B-A3B (Gao et al., 2026), GLM-4.1V-9B-Thinking (V-Team et al., 2025), GUI-R1-3/7B (Luo et al., 2025), MAI-UI-2B (Zhou et al., 2025), Step-GUI-4B (Yan et al., 2025), and GUI-OWL-1.5-8B (Ye et al., 2025), (II) **proprietary VLMs** coupled with a grounding module, following UGround (Gou et al., 2025) and GUI-Libra (Yang et al., 2026), including GPT-4o (OpenAI, 2024), GPT-4.1, GPT-5-mini, and GPT-5 (OpenAI, 2025). We also incorporate reported or reproduced results from prior studies (Liu et al., 2025b; Yang et al., 2026), and (III) **memory-augmented GUI agents**, including Agent Workflow Memory (AWM) (Wang et al., 2024),

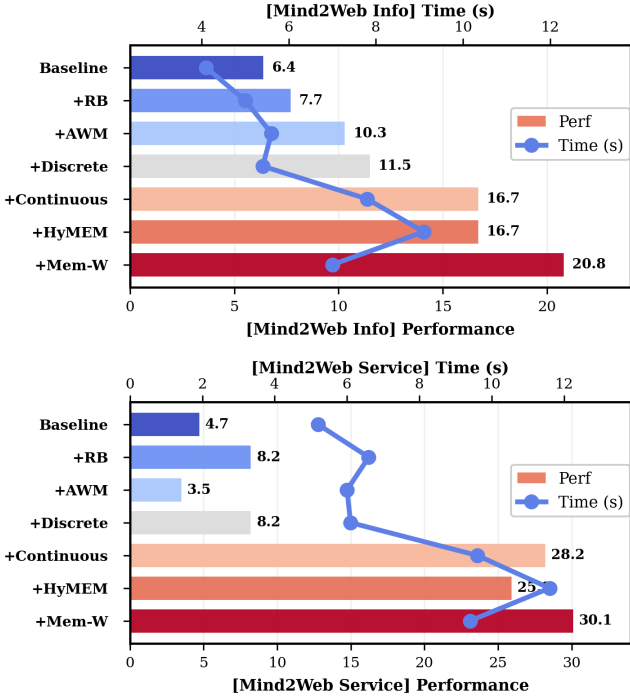


Figure 2: Comparison of memory-enhanced models. “RB” denotes ReasoningBank.

ReasoningBank (Ouyang et al., 2026), CoMEM (Wu et al., 2025b) and HyMEM (Zhu et al., 2026a).

4.2 Main Results

Tables 2 and 3 show that Mem-W consistently improves frozen GUI backbones across both *mobile control* and *web navigation* benchmarks. On AndroidControl-v2, Mem-W improves Qwen3-VL-4B by +13.77/+11.57 on AC-v2-High Pass@1/Pass@4 and by +6.02/+6.80 on AC-v2-Low, while improving UI-Venus-1.5-8B by +7.53/+10.30 and +6.28/+7.03 on the same settings. On GUI-Odyssey, Mem-W further improves Qwen3-VL-4B by +2.18/+2.20 on TM/EM and UI-Venus-1.5-8B by +1.76/+2.60. The gains are especially strong on web navigation: on MMInA, Mem-W raises UI-TARS-1.5-7B, UI-Venus-1.5-8B, and Qwen3-VL-4B by +27.00, +30.00, and +29.00 points, respectively; on Multimodal-Mind2Web, it improves UI-TARS-1.5-7B by +14.71/+15.69, UI-Venus-1.5-8B by +17.65/+20.58, and Qwen3-VL-4B by +8.83/+11.76 on Info/Service. Notably, Mem-W-8B reaches 94.22 Pass@4 on AC-v2-Low, exceeding GPT-5+UGround and GUI-Libra-8B, while also lifting modest backbones to levels competitive with substantially larger open-source GUI models. These results indicate that the bottleneck in long-horizon GUI agency is not merely model scale, but how interaction history and prior experience are represented and exposed to the policy.

4.3 Comparison With Memory-augmented Agents

To further compare Mem-W with existing memory-augmented GUI agents, we evaluate several representative memory designs on Multimodal-Mind2Web in Figure 2. We include three discrete-token memory baselines, namely ReasoningBank (Ouyang et al., 2026), AWM (Wang et al., 2024), and a graph-based discrete memory adapted from HyMEM (Zhu et al., 2026b), which instantiate different explicit memory structures, such as reusable execution templates distilled from successful trajectories or graph-organized experience, but all expose memory to the GUI agent as discrete textual tokens. We also compare with a continuous latent-memory baseline adapted from CoMEM (Wu et al., 2025b), as well as HyMEM (Zhu et al., 2026b), which combines readable symbolic strategies with embedding-based trajectory memory.

Table 3: Performance comparison on MMInA and Mind2Web subsets. The metric used is success rate.

Model	MMInA		Mind2Web	
	Shop	Info	Service	
Open-source Native Models				
Qwen2.5-VL-7B	16.00	9.80	17.65	
GLM-4.1V-9B-Thinking	28.50	10.78	26.47	
GUI-Owl-1.5-8B	5.00	8.82	30.39	
Qwen3-VL-2B	9.50	9.80	23.52	
Qwen3-VL-8B	19.50	13.73	26.47	
Qwen3-VL-32B	36.00	13.73	27.45	
UI-Venus-1.5-30B-A3B	13.50	3.92	8.82	
MAI-UI-2B	2.50	19.41	21.57	
Step-GUI-4B	19.50	12.75	21.56	
Mem-W Models				
UI-TARS-1.5-7B	5.50	5.88	6.86	
↪ Mem-W-7B (Ours)	32.50 (+27.00)	20.59 (+14.71)	22.55 (+15.69)	
UI-Venus-1.5-8B	18.50	5.88	15.69	
↪ Mem-W-8B (Ours)	48.50 (+30.00)	23.53 (+17.65)	36.27 (+20.58)	
Qwen3-VL-4B	11.50	13.72	14.71	
↪ Mem-W-4B (Ours)	40.50 (+29.00)	22.55 (+8.83)	26.47 (+11.76)	

Table 4: Ablation and efficiency analysis on MMINA. Metrics include success rate (Succ.), average number of executed steps per task (#Steps), the percentage of tasks reaching the maximum-step limit (Hit-Max), average runtime per task/step.

Backbone	Setting	Succ.	#Steps	Hit-Max	Time/Task	Time/Step
UI-Venus	Vanilla	18.5	13.0	70.5	83.2	6.4
	w/o Working	47.5	7.7	28.5	82.4	10.7
	w/o Experiential	43.0	10.0	52.1	142.0	14.2
	Full Mem-W	48.5	8.2	42.2	127.9	15.6
Qwen	Vanilla	11.5	15.0	99.0	102.0	6.8
	w/o Working	38.5	8.3	33.0	105.4	12.7
	w/o Experiential	35.5	10.3	52.1	139.1	13.5
	Full Mem-W	40.5	7.1	22.0	117.9	16.6
<i>GUI Agent Baselines</i>						
GLM-4.1V-9B-Thinking		28.5	10.5	53.8	180.6	17.2
Qwen3-VL-32B		36.0	12.7	59.0	125.7	9.9
Step-GUI-4B		19.5	2.5	5.5	32.3	12.9
UI-TARS-1.5-7B		5.5	13.3	75.5	81.1	6.1
UI-Venus-1.5-30B-A3B		13.5	9.6	32.0	119.0	12.4

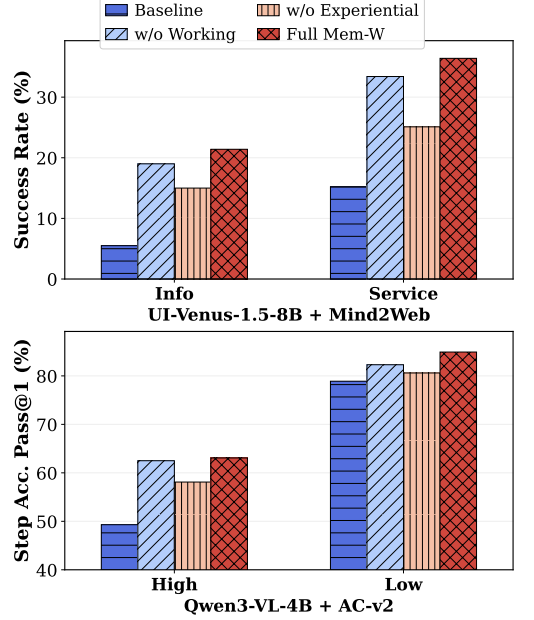


Figure 3: Inference-time ablation.

Results. As shown in Figure 2, Mem-W achieves the strongest performance among all memory-augmented agents on both Multimodal-Mind2Web subsets. On the Info subset, Mem-W improves the baseline from 6.4 to 20.8, outperforming both CoMEM (16.7) and HyMEM (16.7). On the Service subset, Mem-W reaches 30.1, improving over the baseline by +25.4 points and surpassing Continuous memory (28.2) and HyMEM (25.9). Why can Mem-W outperform CoMEM and HyMEM, even though they also use latent embeddings as GUI memory carriers? We attribute this advantage to two factors: first, their working memory remains relatively coarse, largely relying on truncated nearest-neighbor screenshots, whereas Mem-W dynamically distills historical trajectories into compact, decision-relevant latent summaries (Equation (8)); second, their training signal is mainly expert-trajectory imitation, while Mem-W introduces outcome-aware supervision (Equation (12)), enabling memory to preserve long-horizon procedural evidence that is actually tied to task success. In terms of efficiency, Mem-W also remains lightweight: it is faster than HyMEM and CoMEM on Mind2Web, while delivering higher performance.

4.4 Ablation Study

We analyze the contribution of each memory source by selectively removing it from the latent context in Equation (9): *w/o Working* sets $\mathbf{Z}_t^{\text{work}} = \emptyset$, leaving only retrieved experiential memory and the local raw context, while *w/o Experiential* sets $\mathbf{Z}_t^{\text{proc}} = \emptyset$, leaving compressed in-episode history and the current GUI context. As shown in Figure 3 and Table 4, the full Mem-W configuration consistently performs best across backbones and benchmarks: on MMINA, it improves UI-Venus from 18.5 to 48.5 and Qwen from 11.5 to 40.5, while reducing Hit-Max (the rate of hitting max step limit) from 70.5% to 42.2% and from 99% to 22%, respectively. These results suggest that working memory and experiential memory are complementary rather than redundant: the former preserves unfinished in-session progress, while the latter supplies reusable procedural evidence from prior trajectories.

4.5 Efficiency Analysis

While constructing experiential and working memory requires an additional compression step, the overall runtime remains within the same practical range as representative GUI agents, and the added computation is accompanied by substantially more reliable task completion. As shown in Table 4, full Mem-W achieves the best success rate among all compared settings, while reducing the tendency to exhaust the maximum-step budget, indicating that latent memory helps the agent reach successful trajectories more decisively rather than merely extending interaction length. Compared with stronger open-source baselines, Mem-W also provides a favorable accuracy-efficiency trade-off: for

example, it surpasses GLM-4.1V-9B-Thinking and Qwen3-VL-32B in success rate while maintaining comparable or lower task-level runtime. These results suggest that the memory compressor introduces a controlled and worthwhile computational cost.

4.6 Framework Analysis

Beyond aggregate performance, we further analyze how Mem-W behaves under two core design factors of the latent-memory framework. We first study the impact of the number of retrieved trajectories, which controls how much experiential evidence is exposed to the compressor at inference time. We then vary the size of the external memory bank $\mathcal{B}$ to examine whether a richer trajectory reservoir translates into stronger downstream GUI performance.

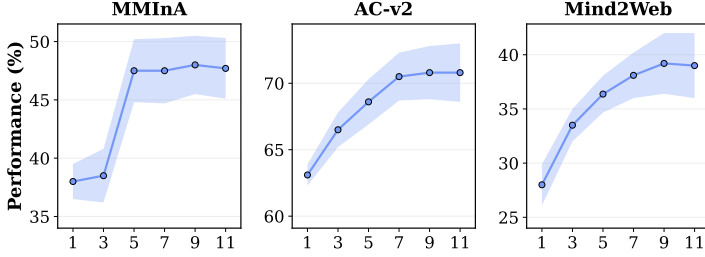


Figure 4: Impact of the number of retrieved trajectories. We vary the retrieval budget M in Equation (5) and report the corresponding performance. The model is UI-Venus.

Impact of Retrieved Trajectories. Figure 4 shows the effect of varying the retrieval budget M . Across MMInA, AC-v2, and Mind2Web, performance generally improves as more trajectories are retrieved, indicating that the procedural memory block $\mathbf{Z}_t^{\text{proc}}$ benefits from richer experiential evidence. On MMInA, performance rises from 38.0 at $M = 1$ to 47.5 at $M = 5$, and remains stable thereafter. On AC-v2, the score increases from 63.1 to 70.8 as M grows from 1 to 9, while Mind2Web improves from 28.0 to 39.2 over the same range. The gains gradually saturate for larger M , suggesting that Mem-W can exploit additional trajectories, but excessive retrieval provides diminishing returns once the most relevant procedural evidence has been covered.

Impact of Memory Bank Size. Table 5 studies how the scale of the external trajectory bank $\mathcal{B}$ affects latent procedural memory. Across all four settings, enlarging $\mathcal{B}$ from 10K to 50K brings clear and consistent gains: Qwen3-VL-4B improves by +5.00 on MMInA and +2.94 on Mind2Web, while UI-Venus improves by +13.50 on MMInA and +8.83 on Mind2Web. The effect is especially pronounced for UI-Venus, where performance rises from 37.00 to 50.50 on MMInA and from 29.41 to 38.24 on Mind2Web, showing that Mem-W can continue to benefit from a richer experiential reservoir at larger scales. Even for Qwen3-VL-4B, the gains remain steady as the bank grows, with MMInA increasing from 35.00 at 10K to 39.50 at 30K and further to 40.00 at 50K. These results suggest that Mem-W can effectively exploit larger trajectory banks, with the learned compressor transforming additional retrieved evidence into compact and informative latent memory.

Table 5: Impact of memory bank size. We vary the number of trajectories stored in $\mathcal{B}$ and evaluate whether larger experiential reservoirs improve latent procedural memory.

Model	Bench	10K	20K	30K	50K
Qwen	MMInA	35.00	38.00	39.50	40.00
Qwen	Mind2Web	25.49	26.47	27.45	28.43
UI-Venus	MMInA	37.00	43.00	47.50	50.50
UI-Venus	Mind2Web	29.41	33.33	36.27	38.24

4.7 Case Study

We provide qualitative case studies to illustrate how Mem-W uses latent memory in long-horizon GUI interaction. Figure 5 and Figure 6 show web and mobile examples, respectively, where the agent must preserve task constraints, intermediate progress, and executable action state across multiple steps. Figure 7 further visualizes the retrieved trajectories for the mobile case, showing how experiential memory complements in-session working memory with reusable procedural evidence.

5 Conclusion

This work introduced Mem-W, a framework for latent-memory-native GUI agents that unifies working memory and experiential memory in a single continuous context space, replacing prescribed memory taxonomies with a shared, end-to-end learnable trajectory-to-latent compressor. By projecting both in-session history and cross-session experience through the same mechanism, Mem-W lets task-relevant memory structure emerge from training rather than manual design. Experiments across four web and mobile benchmarks show that this unified latent interface consistently improves diverse backbones and outperforms agents equipped with hand-engineered memory architectures. More broadly, these results suggest that latent context can be a practical foundation component for future GUI agent architectures: a common substrate where perception, memory, procedural experience, and ongoing task state can be organized in the same machine-native form used by the policy itself. We hope this perspective opens a path toward interactive multimodal agents whose long-horizon competence is built not around externally prescribed memory modules, but around learned latent context interfaces.

Contributions

- **Core Contributors:** Guibin Zhang, Yaohui Ling
- **Contributors:** Fanci Meng
- **Corresponding Authors:** Kun Wang, Shuicheng Yan

If you have any questions regarding the code, paper details, or other aspects of this work, you are very welcome to contact the authors at guibinz@outlook.com or via raising a [Github](#) issue.

References

- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024. URL <https://arxiv.org/abs/2402.14740>.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- Xinghao Chen, Anhao Zhao, Heming Xia, Xuan Lu, Hanlin Wang, Yanjun Chen, Wei Zhang, Jian Wang, Wenjie Li, and Xiaoyu Shen. Reasoning beyond language: A comprehensive survey on latent chain-of-thought reasoning, 2025. URL <https://arxiv.org/abs/2505.16782>.
- Weihua Cheng, Junming Liu, Yifei Sun, Botian Shi, Yirong Chen, and Ding Wang. Mga: Memory-driven gui agent for observation-centric interaction, 2026. URL <https://arxiv.org/abs/2510.24168>.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2026. URL <https://arxiv.org/abs/2508.06433>.
- Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, Beitong Zhou, et al. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*, 2026.
- Xinzge Gao, Chuanrui Hu, Bin Chen, and Teng Li. Chain-of-memory: Enhancing gui agents for cross-application navigation, 2025. URL <https://arxiv.org/abs/2506.18158>.
- Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents, 2025. URL <https://arxiv.org/abs/2410.05243>.
- Yubo Hou, Zhisheng Chen, Tao Wan, and Zengchang Qin. Flashmem: Distilling intrinsic latent memory via computation reuse, 2026. URL <https://arxiv.org/abs/2601.05505>.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiabin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2026. URL <https://arxiv.org/abs/2512.13564>.
- Kung-Hsiang Huang, Haoyi Qiu, Yutong Dai, Caiming Xiong, and Chien-Sheng Wu. Gui-kv: Efficient gui agents via kv cache with spatio-temporal awareness, 2025. URL <https://arxiv.org/abs/2510.00536>.

- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models, 2023. URL <https://arxiv.org/abs/2301.12597>.
- Runze Li, Yuwen Zhai, Bo Xu, LiWu Xu, Nian Shi, Wei Zhang, Ran Lin, and Liang Wang. Echotrail-gui: Building actionable memory for gui agents via critic-guided self-exploration, 2026a. URL <https://arxiv.org/abs/2512.19396>.
- Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems*, 37:92130–92154, 2024.
- Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. Deepagent: A general reasoning agent with scalable toolsets, 2026b. URL <https://arxiv.org/abs/2510.21618>.
- Jiaqi Liu, Zipeng Ling, Shi Qiu, Yanqing Liu, Siwei Han, Peng Xia, Haoqin Tu, Zeyu Zheng, Cihang Xie, Charles Fleming, Mingyu Ding, and Huaxiu Yao. Omni-simplemem: Autoresearch-guided discovery of lifelong multimodal agent memory, 2026. URL <https://arxiv.org/abs/2604.01007>.
- Junming Liu, Yifei Sun, Weihua Cheng, Haodong Lei, Yirong Chen, Licheng Wen, Xuemeng Yang, Daocheng Fu, Pinlong Cai, Nianchen Deng, Yi Yu, Shuyue Hu, Botian Shi, and Ding Wang. Memverse: Multimodal memory for lifelong learning agents, 2025a. URL <https://arxiv.org/abs/2512.03627>.
- Zhaoyang Liu, JingJing Xie, Zichen Ding, Zehao Li, Bowen Yang, Zhenyu Wu, Xuehui Wang, Qiushi Sun, Shi Liu, Weiyun Wang, et al. Scalecua: Scaling open-source computer use agents with cross-platform data. *arXiv preprint arXiv:2509.15221*, 2025b.
- Quanfeng Lu, Wenqi Shao, Zitao Liu, Lingxiao Du, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, and Ping Luo. Guidyssey: A comprehensive dataset for cross-app gui navigation on mobile devices, 2025. URL <https://arxiv.org/abs/2406.08451>.
- Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025.
- Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, Xintong Li, Jing Shi, Hongjie Chen, Viet Dac Lai, Zhouhang Xie, Sungchul Kim, Ruiyi Zhang, Tong Yu, Mehrab Tanjim, Nesreen K. Ahmed, Puneet Mathur, Seunghyun Yoon, Lina Yao, Branislav Kveton, Jihyung Kil, Thien Huu Nguyen, Trung Bui, Tianyi Zhou, Ryan A. Rossi, and Franck Dernoncourt. GUI agents: A survey. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22522–22538, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1158. URL <https://aclanthology.org/2025.findings-acl.1158/>.
- OpenAI. Introducing gpt-4o. Available at: <https://openai.com/index/hello-gpt-4o>, 2024.
- OpenAI. GPT-5 is here — openai.com. <https://openai.com/gpt-5/>, 2025.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2026. URL <https://arxiv.org/abs/2509.25140>.

- Wenbo Pan, Shujie Liu, Xiangyang Zhou, Shiwei Zhang, Wanlu Shi, Mirror Xu, and Xiaohua Jia. M*: Every task deserves its own memory harness. *arXiv preprint arXiv:2604.11811*, 2026.
- ByteDance Seed. Ui-tars-1.5. <https://seed-tars.com/1.5>, 2025.
- Yibo Shi, Jungang Li, Linghao Zhang, Zihao Dongfang, Biao Wu, Sicheng Tao, Yibo Yan, Chenxi Qin, Weiting Liu, Zhixin Lin, Hanqian Li, Yu Huang, Song Dai, Yonghua Hei, Yue Ding, Xiang Li, Shikang Wang, Chengdong Xu, Jingqi Liu, Xueying Ma, Zhiwen Zheng, Xiaofei Zhang, Bincheng Wang, Nichen Yang, Jie Wu, Lihua Tian, Chen Li, and Xuming Hu. Androtmem: From interaction trajectories to anchored memory in long-horizon gui agents, 2026. URL <https://arxiv.org/abs/2603.18429>.
- Yurun Song, Jiong Yin, Rongjunchen Zhang, and Ian G. Harris. Compress to focus: Efficient coordinate compression for policy optimization in multi-turn gui agents, 2026. URL <https://arxiv.org/abs/2601.11631>.
- Shulin Tian, Ziniu Zhang, Liang-Yu Chen, and Ziwei Liu. Mmina: Benchmarking multihop multimodal internet agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 13682–13697, 2025.
- V-Team, Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, Shuaiqi Duan, Weihan Wang, Yan Wang, Yean Cheng, Zehai He, Zhe Su, Zhen Yang, Ziyang Pan, Aohan Zeng, Baoxu Wang, Bin Chen, Boyan Shi, Changyu Pang, Chenhui Zhang, Da Yin, Fan Yang, Guoqing Chen, Jiazheng Xu, Jiale Zhu, Jiali Chen, Jing Chen, Jinhao Chen, Jinghao Lin, Jinjiang Wang, Junjie Chen, Leqi Lei, Letian Gong, Leyi Pan, Mingdao Liu, Mingde Xu, Mingzhi Zhang, Qinkai Zheng, Sheng Yang, Shi Zhong, Shiyu Huang, Shuyuan Zhao, Siyan Xue, Shangqin Tu, Shengbiao Meng, Tianshu Zhang, Tianwei Luo, Tianxiang Hao, Tianyu Tong, Wenkai Li, Wei Jia, Xiao Liu, Xiaohan Zhang, Xin Lyu, Xinyue Fan, Xuancheng Huang, Yanling Wang, Yadong Xue, Yanfeng Wang, Yanzi Wang, Yifan An, Yifan Du, Yiming Shi, Yiheng Huang, Yilin Niu, Yuan Wang, Yuanchang Yue, Yuchen Li, Yutao Zhang, Yuting Wang, Yu Wang, Yuxuan Zhang, Zhao Xue, Zhenyu Hou, Zhengxiao Du, Zihan Wang, Peng Zhang, Debing Liu, Bin Xu, Juanzi Li, Minlie Huang, Yuxiao Dong, and Jie Tang. Glm-4.5v and glm-4.1v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning, 2025. URL <https://arxiv.org/abs/2507.01006>.
- Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. M+: Extending memoryllm with scalable long-term memory, 2025a. URL <https://arxiv.org/abs/2502.00592>.
- Yu Wang, Xinshuang Liu, Xiusi Chen, Sean O’Brien, Junda Wu, and Julian McAuley. Self-updatable large language models by integrating context into model parameters, 2025b. URL <https://arxiv.org/abs/2410.00487>.
- Ziwei Wang, Leyang Yang, Xiaoxuan Tang, Sheng Zhou, Dajun Chen, Wei Jiang, and Yong Li. History-aware reasoning for gui agents, 2025c. URL <https://arxiv.org/abs/2511.09127>.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024. URL <https://arxiv.org/abs/2409.07429>.
- Wenyi Wu, Zixuan Song, Kun Zhou, Yifei Shao, Zhiting Hu, and Biwei Huang. Towards general continuous memory for vision-language models, 2025a. URL <https://arxiv.org/abs/2505.17670>.
- Wenyi Wu, Kun Zhou, Ruoxin Yuan, Vivian Yu, Stephen Wang, Zhiting Hu, and Biwei Huang. Auto-scaling continuous memory for gui agent, 2025b. URL <https://arxiv.org/abs/2510.09038>.

- Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. From human memory to ai memory: A survey on memory mechanisms in the era of llms, 2025c. URL <https://arxiv.org/abs/2504.15965>.
- Xun Xu. G-memllm: Gated latent memory augmentation for long-context reasoning in large language models, 2026. URL <https://arxiv.org/abs/2602.00015>.
- Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot: Soft chain-of-thought for efficient reasoning with llms, 2025a. URL <https://arxiv.org/abs/2502.12134>.
- Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot++: Test-time scaling with soft chain-of-thought reasoning, 2025b. URL <https://arxiv.org/abs/2505.11484>.
- Zhou Xu, Bowen Zhou, Qi Wang, Shuwen Feng, and Jingyu Xiao. Spatio-temporal token pruning for efficient high-resolution gui agents, 2026. URL <https://arxiv.org/abs/2602.23235>.
- Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, Shiliang Yang, Zhirui Wang, Brian Li, Kang An, Chenyang Li, Lei Lei, Mengmeng Duan, Danxun Liang, Guodong Liu, Hang Cheng, Hao Wu, Jie Dong, Junhao Huang, Mei Chen, Renjie Yu, Shunshan Li, Xu Zhou, Yiting Dai, Yineng Deng, Yingdan Liang, Zelin Chen, Wen Sun, Chengxu Yan, Chunqin Xu, Dong Li, Fengqiong Xiao, Guanghao Fan, Guopeng Li, Guozhen Peng, Hongbing Li, Hang Li, Hongming Chen, Jingjing Xie, Jianyong Li, Jingyang Zhang, Jiaju Ren, Jiayu Yuan, Jianpeng Yin, Kai Cao, Liang Zhao, Liguang Tan, Liying Shi, Mengqiang Ren, Min Xu, Manjiao Liu, Mao Luo, Mingxin Wan, Na Wang, Nan Wu, Ning Wang, Peiyao Ma, Qingzhou Zhang, Qiao Wang, Qinlin Zeng, Qiong Gao, Qiongyao Li, Shangwu Zhong, Shuli Gao, Shaofan Liu, Shisi Gao, Shuang Luo, Xingbin Liu, Xiaojia Liu, Xiaojie Hou, Xin Liu, Xuanti Feng, Xuedan Cai, Xuan Wen, Xianwei Zhu, Xin Liang, Xin Liu, Xin Zhou, Yifan Sui, Yingxiu Zhao, Yukang Shi, Yunfang Xu, Yuqing Zeng, Yixun Zhang, Zejia Weng, Zhonghao Yan, Zhiguo Huang, Zhuoyu Wang, Zihan Yan, Zheng Ge, Jing Li, Yibo Zhu, Binxing Jiao, Xiangyu Zhang, and Daxin Jiang. Step-gui technical report, 2025. URL <https://arxiv.org/abs/2512.15431>.
- Rui Yang, Qianhui Wu, Zhaoyang Wang, Hanyang Chen, Ke Yang, Hao Cheng, Huaxiu Yao, Baoling Peng, Huan Zhang, Jianfeng Gao, et al. Gui-libra: Training native gui agents to reason and act with action-aware supervision and partially verifiable rl. *arXiv preprint arXiv:2602.22190*, 2026.
- Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. Mobile-agent-v3: Fundamental agents for gui automation, 2025. URL <https://arxiv.org/abs/2508.15144>.
- Xinlei Yu, Zhangquan Chen, Yongbo He, Tianyu Fu, Cheng Yang, Chengming Xu, Yue Ma, Xiaobin Hu, Zhe Cao, Jie Xu, Guibin Zhang, Jiale Tao, Jiayi Zhang, Siyuan Ma, Kaituo Feng, Haojie Huang, Youxing Li, Ronghao Chen, Huacan Wang, Chenglin Wu, Zikun Su, Xiaogang Xu, Kelu Yao, Kun Wang, Chen Gao, Yue Liao, Ruqi Huang, Tao Jin, Cheng Tan, Jiangning Zhang, Wenqi Ren, Yanwei Fu, Yong Liu, Yu Wang, Xiangyu Yue, Yu-Gang Jiang, and Shuicheng Yan. The latent space: Foundation, evolution, mechanism, ability, and outlook, 2026a. URL <https://arxiv.org/abs/2604.02029>.
- Xinlei Yu, Chengming Xu, Zhangquan Chen, Bo Yin, Cheng Yang, Yongbo He, Yihao Hu, Jiangning Zhang, Cheng Tan, Xiaobin Hu, and Shuicheng Yan. Dual latent memory for visual multi-agent system, 2026b. URL <https://arxiv.org/abs/2602.00471>.

- Xinlei Yu, Chengming Xu, Guibin Zhang, Zhangquan Chen, Yudong Zhang, Yongbo He, Peng-Tao Jiang, Jiangning Zhang, Xiaobin Hu, and Shuicheng Yan. Vismem: Latent vision memory unlocks potential of vision-language models, 2026c. URL <https://arxiv.org/abs/2511.11007>.
- Qianhao Yuan, Jie Lou, Zichao Li, Jiawei Chen, Yaojie Lu, Hongyu Lin, Le Sun, Debing Zhang, and Xianpei Han. Memsearcher: Training llms to reason, search and manage memory via end-to-end reinforcement learning, 2025. URL <https://arxiv.org/abs/2511.02805>.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems, 2025. URL <https://arxiv.org/abs/2512.18746>.
- Guibin Zhang, Muxin Fu, and Shuicheng YAN. Memgen: Weaving generative latent memory for self-evolving agents. In *The Fourteenth International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=vI56m4Iu4e>.
- Zeyu Zhang, Rui Li, Xiaoyan Zhao, Yang Zhang, Wenjie Wang, Xu Chen, and Tat-Seng Chua. Nextmem: Towards latent factual memory for llm-based agents, 2026b. URL <https://arxiv.org/abs/2603.15634>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144>.
- Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v (ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614*, 2024.
- Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesatapornwongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. Actionengine: From reactive to programmatic gui agents via state machine memory, 2026. URL <https://arxiv.org/abs/2602.20502>.
- Bowen Zhou, Zhou Xu, Wanli Li, Jingyu Xiao, and Haoqian Wang. Efficient long-horizon gui agents via training-free kv cache compression, 2026. URL <https://arxiv.org/abs/2603.00188>.
- Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, and Steven Hoi. Mai-ui technical report: Real-world centric foundation gui agents, 2025. URL <https://arxiv.org/abs/2512.22047>.
- Rui-Jie Zhu, Tianhao Peng, Tianhao Cheng, Xingwei Qu, Jinfa Huang, Dawei Zhu, Hao Wang, Kaiwen Xue, Xuanliang Zhang, Yong Shan, Tianle Cai, Taylor Kergan, Assel Kembay, Andrew Smith, Chenghua Lin, Binh Nguyen, Yuqi Pan, Yuhong Chou, Zefan Cai, Zhenhe Wu, Yongchi Zhao, Tianyu Liu, Jian Yang, Wangchunshu Zhou, Chujie Zheng, Chongxuan Li, Yuyin Zhou, Zhoujun Li, Zhaoxiang Zhang, Jiaheng Liu, Ge Zhang, Wenhao Huang, and Jason Eshraghian. A survey on latent reasoning, 2025. URL <https://arxiv.org/abs/2507.06203>.
- Sibo Zhu, Wenyi Wu, Kun Zhou, Stephen Wang, and Biwei Huang. Hybrid self-evolving structured memory for gui agents. *arXiv preprint arXiv:2603.10291*, 2026a.
- Sibo Zhu, Wenyi Wu, Kun Zhou, Stephen Wang, and Biwei Huang. Hybrid self-evolving structured memory for gui agents, 2026b. URL <https://arxiv.org/abs/2603.10291>.

A Training Details

This appendix provides the full training procedure for Mem-W, expanding on the two-stage recipe described in Section 3.4. We restate the key convention: the GUI agent Π_θ (encoder $\mathbf{G}_\theta$, decoder, and all associated weights) is *frozen* throughout both stages. Only the compressor parameters $\phi = \{\mathbf{Q}, \text{QFormer}_\phi, \mathbf{P}_\phi, \gamma, \mathbf{b}^{\text{proc}}, \mathbf{b}^{\text{work}}\}$ receive gradient updates.

A.1 Stage 1: Self-Distillation

Data construction. We sample a trajectory $\tau = (x, s_{1:T}, y)$ from a demonstration dataset $\mathcal{D}$ and uniformly draw a timestep t such that the expired prefix $\bar{s}_t = s_{1:\max(0, t-L-1)}$ is non-empty (i.e. $t > L + 1$), so that working-memory compression is exercised. The raw context window is $\rho_t = s_{\max(1, t-L):t-1}$. To also exercise the procedural-memory path, we retrieve M trajectories from the external bank $\mathcal{B}$ using Equation (5), yielding the retrieved index set $\mathcal{I}_t$.

Teacher context. The teacher branch feeds the same frozen agent an extended raw window of L' steps ($L' \gg L$):

$$\chi_t^{\text{raw}} = (x, s_{\max(1, t-L'):t-1}, o_t). \quad (13)$$

Because Π_θ is frozen and χ_t^{raw} contains no learnable latent tokens, the teacher output $\Pi_\theta(\cdot \mid \chi_t^{\text{raw}})$ is treated as a fixed target (gradients are stopped via $\text{sg}[\cdot]$). Note that the teacher observes a larger but still finite window; it is not assumed to see the entire trajectory.

Student context. The student branch constructs the weaved input $\mathbf{U}_t$ from Equation (9), with both procedural and working memory active. The procedural-memory block is built by compressing the retrieved trajectories with their outcome labels,

$$\mathbf{Z}_t^{\text{proc}} = [C_\phi(s_{1:T_i}^i, y_i)]_{i \in \mathcal{I}_t}, \quad (14)$$

while the working-memory block $\mathbf{Z}_t^{\text{work}}$ is built by partitioning the expired prefix $\bar{s}_t$ into non-overlapping chunks of W steps and compressing each chunk with the unknown-outcome marker $\emptyset$ as in Equation (8). The student input is therefore

$$\mathbf{U}_t^{\text{s1}} = [\mathbf{Z}_t^{\text{proc}} + \mathbf{1}_{MK}(\mathbf{b}^{\text{proc}})^\top; \mathbf{Z}_t^{\text{work}} + \mathbf{1}_{JK}(\mathbf{b}^{\text{work}})^\top; \mathbf{E}_\theta(x, o_t, \rho_t)]. \quad (15)$$

Loss. For each sample (τ, t) the stage-one loss is

$$\mathcal{L}_{\text{sd}}(\phi; \tau, t) = \underbrace{\ell_{\text{gui}}(\Pi_\theta(\cdot \mid \mathbf{U}_t^{\text{s1}}), a_t^*)}_{\text{action loss}} + \lambda \underbrace{D_{\text{KL}}(\text{sg}[\Pi_\theta(\cdot \mid \chi_t^{\text{raw}})] \parallel \Pi_\theta(\cdot \mid \mathbf{U}_t^{\text{s1}}))}_{\text{distillation loss}}, \quad (16)$$

where a_t^* is the ground-truth action at step t and λ is a balancing coefficient.

The action loss ℓ_{gui} is the standard next-token cross-entropy over the action token sequence: if the target action serialization is $(a_t^{(1)}, \dots, a_t^{(S)})$, then

$$\ell_{\text{gui}}(\Pi_\theta(\cdot \mid \mathbf{U}_t^{\text{s1}}), a_t^*) = -\frac{1}{S} \sum_{s=1}^S \log p_{\theta, \phi}(a_t^{(s)} \mid \mathbf{U}_t^{\text{s1}}, a_t^{(1:s-1)}), \quad (17)$$

where $p_{\theta, \phi}$ denotes the next-token probability of the frozen agent conditioned on the latent-augmented context. Since θ is frozen, gradients from both losses reach learnable parameters only through the procedural and working latent blocks produced by C_ϕ .

A.2 Stage 2: RLOO-Based Outcome-Aware Optimization

Stage 2 keeps both procedural and working memory active and further optimizes the compressor with environment feedback. The agent backbone θ remains frozen; gradients reach ϕ through the differentiable latent tokens in the weaved context $\mathbf{U}_t$.

Rollout and reward. For each task instruction x drawn from a task distribution $p(x)$, we sample G independent rollout trajectories from the current policy:

$$\tau^{(g)} = (x, s_{1:T^{(g)}}^{(g)}, y^{(g)}), \quad g = 1, \dots, G, \quad \tau^{(g)} \sim \pi_{\theta, \phi}, \quad (18)$$

where each trajectory is produced by conditioning the frozen agent on the latent-augmented context $\mathbf{U}_t^{(g)}$ constructed with the current compressor C_ϕ . The terminal reward is binary: $R(\tau^{(g)}) \in \{0, 1\}$, with 1 indicating task success.

RLOO advantage estimation. Following RLOO estimator (Ahmadian et al., 2024), we construct an unbiased, low-variance advantage for each rollout by using the other $G - 1$ rollouts to the same instruction as a baseline. For the g -th rollout, the advantage is

$$\hat{A}^{(g)} = R(\tau^{(g)}) - \frac{1}{G-1} \sum_{\substack{g'=1 \\ g' \neq g}}^G R(\tau^{(g')}). \quad (19)$$

This per-instruction baseline is parameter-free and does not require a learned value function. For computational efficiency, we rewrite it as

$$\hat{A}^{(g)} = \frac{G}{G-1} \left(R(\tau^{(g)}) - \bar{R}_x \right), \quad \bar{R}_x = \frac{1}{G} \sum_{g'=1}^G R(\tau^{(g')}), \quad (20)$$

where $\bar{R}_x$ is the mean reward over all G rollouts for instruction x and is computed once per instruction.

Per-trajectory policy gradient. Because the reward is trajectory-level (success or failure of the entire episode), we assign the same advantage $\hat{A}^{(g)}$ to every timestep within rollout $\tau^{(g)}$. The per-trajectory policy-gradient loss for the g -th rollout is

$$\mathcal{L}_{\text{pg}}^{(g)}(\phi) = -\hat{A}^{(g)} \cdot \frac{1}{T^{(g)}} \sum_{t=1}^{T^{(g)}} \log \Pi_\theta(a_t^{(g)} \mid \mathbf{U}_t^{(g)}), \quad (21)$$

where the log-probability is summed over the action tokens at each step and averaged over timesteps.

KL regularization. To prevent the compressor from drifting too far from the stage-one solution, we add a KL penalty against a frozen reference policy π_{ref} . The reference policy uses the stage-one compressor $C_{\phi_{\text{ref}}}$ (with parameters frozen after stage 1) to construct a reference context $\mathbf{U}_t^{\text{ref}}$, while the agent decoder Π_θ remains the same. The per-step KL term is

$$D_t^{(g)} = D_{\text{KL}}\left(\Pi_\theta(\cdot \mid \mathbf{U}_t^{(g)}) \parallel \Pi_\theta(\cdot \mid \mathbf{U}_t^{\text{ref},(g)})\right), \quad (22)$$

where $\mathbf{U}_t^{\text{ref},(g)}$ is constructed identically to $\mathbf{U}_t^{(g)}$ except that the latent tokens are produced by the frozen $C_{\phi_{\text{ref}}}$ instead of the current C_ϕ . Because $\mathbf{U}_t^{\text{ref},(g)}$ involves no learnable parameters, gradients of $D_t^{(g)}$ flow only into ϕ through $\mathbf{U}_t^{(g)}$.

Table 6: Web Training Data

Domain	Success	Failure	Total	Success Rate (%)
academic	1246	6579	7825	15.92
education	1045	6536	7581	13.78
entertainment	297	1817	2114	14.05
finance	692	3907	4599	15.05
food	835	4840	5675	14.71
government	1417	10491	11908	11.90
health	1380	9447	10827	12.75
news	600	3386	3986	15.05
service	861	5530	6391	13.47
shopping	612	4634	5246	11.67
social	415	1666	2081	19.94
tech	1505	7231	8736	17.23
travel	271	4287	4558	5.95
Overall Summary	11176	70351	81527	13.71

Combined objective. The full stage-two objective, averaged over a batch of instructions and their rollouts, is

$$\mathcal{L}_{s2}(\phi) = \mathbb{E}_{x \sim p(x)} \left[\frac{1}{G} \sum_{g=1}^G \left(\mathcal{L}_{pg}^{(g)}(\phi) + \frac{\beta}{T^{(g)}} \sum_{t=1}^{T^{(g)}} D_t^{(g)} \right) \right], \quad (23)$$

where β is the KL penalty coefficient. Expanding the policy-gradient and KL terms together, the per-rollout contribution can equivalently be written as

$$\frac{1}{T^{(g)}} \sum_{t=1}^{T^{(g)}} \left(-\hat{A}^{(g)} \log \Pi_{\theta}(a_t^{(g)} | \mathbf{U}_t^{(g)}) + \beta D_{KL}(\Pi_{\theta}(\cdot | \mathbf{U}_t^{(g)}) \parallel \Pi_{\theta}(\cdot | \mathbf{U}_t^{\text{ref},(g)})) \right), \quad (24)$$

which directly corresponds to Equation (12) in the main text, with $\hat{R}(\tau)$ instantiated by the RLOO advantage $\hat{A}^{(g)}$.

At each step t , the frozen decoder Π_{θ} receives $\mathbf{U}_t^{(g)}$ as input embeddings. The latent blocks $\mathbf{Z}_t^{\text{proc}}$ and $\mathbf{Z}_t^{\text{work}}$ inside $\mathbf{U}_t^{(g)}$ are differentiable outputs of the compressor C_{ϕ} . When computing $\nabla_{\phi} \mathcal{L}_{s2}$, the gradient passes from the scalar loss through the frozen decoder’s attention layers (which are applied but not updated) into the latent tokens, and then into the QFormer and projection head of C_{ϕ} . The retrieval operation (Top-M over frozen encoder outputs) is discrete and contributes no gradient; only the compression of the retrieved segments is differentiable. Similarly, the reference context $\mathbf{U}_t^{\text{ref},(g)}$ is fully detached, so the KL term’s gradient flows only through the current compressor’s output.

B Experiment Details

B.1 Training Dataset

Web Training Dataset. The Web training dataset is mainly from CoMEM (Wu et al., 2025b); it covers 13 domains, including academic, education, finance, government, health, shopping, tech, and travel. Each sample is labeled as either successful or failed according to its task execution outcome. In total, the Web training set contains 81,527 samples, consisting of 11,176 successful trajectories and 70,351 failed trajectories. During training, we use the successful trajectories as training examples.

Table 7: Web Memory Bank

Domain	Success	Failure	Total	Success Rate (%)
academic	2491	13158	15649	15.92
education	2089	13072	15161	13.78
entertainment	593	3634	4227	14.03
finance	1384	7814	9198	15.05
food	1670	9680	11350	14.71
government	2834	20982	23816	11.90
health	2760	18894	21654	12.75
news	1199	6772	7971	15.04
service	1722	11060	12782	13.47
shopping	1223	9268	10491	11.66
social	830	3332	4162	19.94
tech	3010	14462	17472	17.23
travel	541	8574	9115	5.94
Overall Summary	22346	140702	163048	13.71

Table 8: Mobile Training Data

Domain	Success	Failure	Total	Success Rate (%)
utility	832	2	834	99.76
social communication	532	0	532	100.00
media services	517	3	520	99.42
information handling	335	52	387	86.56
cross-app workflow	232	19	251	92.43
e-commerce	41	47	88	46.59
Overall Summary	2489	123	2612	95.29

Web Memory Bank. The web memory bank is also sampled from (Wu et al., 2025b); it is designed to store reusable task-execution trajectories and domain-specific knowledge. It contains a total of 22,346 successful memories, covering the same 13 domains as the Web training set. These memories provide useful references for planning, retrieval, and decision-making when the model encounters similar Web-based tasks.

Mobile Training Dataset. The Mobile training dataset is drawn from GUI-Odyssey (Lu et al., 2025), using the official training split provided at https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey/blob/main/splits/random_split.json. We emphasize that training is conducted only on the official training set, while evaluation uses the official test set, ensuring that **there is no overlap or data leakage between training and evaluation**. It covers six categories of mobile tasks, including Utility Operations, Social Posting, Media Services, Data Organization, Cross-App Workflows, and Online Purchasing. The Mobile training set contains 2,610 samples, including 2,486 successful trajectories and 124 failed trajectories. Similar to the Web setting, we use the successful trajectories for training.

Mobile Memory Bank. The mobile memory bank is also constructed from the official GUI-Odyssey training split (Lu et al., 2025). It contains 4,972 successful memories spanning all six categories of mobile tasks, capturing reusable operation patterns for cross-application workflows, information organization, and general mobile interaction. **We emphasize that the memory bank is built exclusively from training-set trajectories and has no overlap with the official test set used for evaluation, thereby avoiding any data leakage.**

Table 9: Mobile Memory Bank

Domain	Success	Failure	Total	Success Rate (%)
utility	1663	5	1668	99.70
social communication	1063	0	1063	100.00
media services	1033	6	1039	99.42
information handling	669	104	773	86.55
cross-app workflow	463	39	502	92.23
e-commerce	81	95	176	46.02
Overall Summary	4972	249	5221	95.23

B.2 Parameter Configuration

During training, we adopt a parameter-efficient fine-tuning strategy based on LoRA, together with DeepSpeed ZeRO-3 for memory optimization and distributed training. The original parameters of both the language model backbone and the visual encoder are frozen, while only the LoRA adapters and the compressor-related modules are updated. This design reduces the overall training cost while preserving the general vision-language capabilities of the pretrained foundation model.

The model is trained using bfloat16 mixed precision, gradient checkpointing, a cosine learning-rate scheduler, and the AdamW optimizer. The learning rate is set to 5×10^{-5} , with a weight decay of 0.1 and a warmup ratio of 0.03. For LoRA, the rank is set to 16, the scaling factor α is set to 32, and the dropout rate is set to 0.05. Under the default configuration, the per-GPU batch size is set to 2. All the experiments are conducted on a server with 8 NVIDIA A800 (80GB) GPUs.

B.3 Evaluation Setup

We evaluated our method on four benchmarks, namely MMinA, Mind2Web, GUI-Odyssey, and Android-Control-v2. As MMinA and Mind2Web share the same interactive GUI environment, we describe their experimental setup and results together. In contrast, the evaluations on GUI-Odyssey and Android-Control-v2 are presented separately.

Interactive GUI Evaluation on MMinA and Mind2Web. During the web evaluation stage, we deploy the agent in an interactive GUI environment and adopt a ReAct-style reasoning-acting paradigm. At each step, the agent receives the current web state as input, including the page screenshot, textual information extracted from the page, the task instruction, and the most recent action history. Based on this information, the model is required to analyze the current state, plan the next operation, and output a structured function call. The action space includes clicking, typing text, pressing keys, scrolling, waiting, and producing the final answer followed by termination.

For each task, we set the maximum number of interaction steps to 15. A task terminates when the agent explicitly outputs a `stop` action or when the maximum step limit is reached.

When continuous memory is enabled, the model is loaded and used for generation through a local Transformers backend. In this mode, the inference parameters are set to `temperature = 0.1`, `top_p = 0.001`, and a maximum generation length of 10,000 new tokens, with KV-cache enabled. If no relevant memory is retrieved, the system falls back to the standard vLLM inference pipeline.

The memory module retrieves historical trajectories that are similar to the current task using a FAISS index. Specifically, embeddings are first normalized with L2 normalization, and retrieval is performed using inner-product similarity. In the actual evaluation, the top five most similar historical experiences are retrieved for each task and

inserted into the system prompt as experience memory, providing demonstrations and contextual references for solving the current task.

Benchmark Evaluation on GUI-Odyssey. During each reasoning step in GUI-Odyssey, the model input consists of the current GUI observation, the task instruction, and the retrieved experience memory.

Since GUI-Odyssey is evaluated in an offline setting, the retrieval process differs from that used in interactive evaluation, where trajectory-level experiences can be directly retrieved during interaction. Instead, during inference, candidate experiences are retrieved from a step-level memory index. For each task, we retrieve the top five most similar historical experiences and incorporate them into the prompt as contextual references.

For generation, we adopt a deterministic decoding strategy. Specifically, the temperature is set to 0.0, top- p is set to 1.0, and the maximum generation length for each single-step action prediction is set to 64 tokens.

Benchmark Evaluation on Android-Control-v2. For AndroidControl-v2, we evaluate models under two instruction settings: High-Level and Low-Level. In the High-Level setting, the model is provided only with the high-level task goal. It must infer the appropriate next action, such as where to click, what to type, whether to navigate back, or whether to scroll, based on the current screenshot and the interaction history. In the Low-Level setting, in addition to the high-level task goal, the model is given the oracle low-level instruction for the current step.

We further evaluate model performance under different numbers of generation attempts. Specifically, we consider two settings: Pass@1 and Pass@4. For Pass@1, a single output is generated for each sample, and the sample is considered successful only if this output is correct. For Pass@4, four candidate outputs are generated for each sample, and the sample is considered successful if any one of the four candidates is correct. We set the temperature to 0.0 for Pass@1 and to 1.0 for Pass@4, with a maximum generation length of 2048 tokens.

Similar to GUI-Odyssey, during inference we retrieve candidate experiences from the step-level memory index. For each task, we retrieve the top-5 most similar historical experiences; the sensitivity analysis of this parameter has been provided in Section 4.6.

B.4 Evaluation Benchmark

MMIna. MMIna is a multimodal, multi-hop, and open-ended benchmark for evaluating web agents in realistic web environments. It emphasizes the ability of an agent to autonomously plan action trajectories in a browser according to user instructions, while jointly leveraging webpage screenshots, HTML content, and multimodal information such as text and images for understanding and decision making.

The *shopping* subset represents a typical e-commerce scenario in MMIna. The tasks all centered around products from OneStopMarket. The subset adopts a discrete web interaction action space, which contains 12 types of high-level actions. These actions cover the major operations required for completing shopping-related tasks in a web environment, including clicking, typing, and hovering over webpage elements; scrolling the page and issuing keyboard commands; as well as browser-level navigation actions such as visiting a URL, moving backward or forward, opening a new tab, closing a tab, and switching page focus, as detailed in Table 10. In addition, the action space includes the `stop [answer]` action, which indicates task completion and returns the final answer.

Agent Inference Prompt. The prompt used during interactive GUI evaluation is dynamically constructed at each step. The system message is loaded from the agent prompt template and filled with the retrieved experience memory and the available tool specifications. The user-side context contains the current webpage screenshot, a generated

page description, and the current task instruction. The simplified prompt template used during evaluation is shown below.

Agent Inference Prompt

SYSTEM INSTRUCTION

You are a GUI automation agent that can interact with web pages and applications using the ReAct (Reasoning and Acting) paradigm.

IMPORTANT: You MUST output your actions in structured JSON format that can be parsed directly. Use the function calling mechanism to execute actions.

Your task is to:

1. Analyze the current state of the page, including numerical labels on web elements.
2. Think through what needs to be done.
3. Determine the appropriate action to take.
4. Output the action in structured JSON format that can be parsed directly.

WORKFLOW GUIDELINES:

- If your previous action is type, then you must click related pages or scroll pages to find the information you need.
- When you need to search for information, directly type your search query, then click the search button.
- After clicking an element, if you need to interact with it further, such as typing, do so immediately.
- Do not repeat the same action multiple times. If something does not work, try a different approach.
- Always use function calling to execute actions. Do not describe actions in plain text.
- If the current page has no results, adjust your search term and try again.
- Pay attention to images, since many questions about shape, color, location, and visual content can be answered from screenshots.

ACTION GUIDELINES:

1. To input text, no need to click the textbox first. Directly type the content. After typing, the system automatically hits the ENTER key.
2. Distinguish between textboxes and search buttons. Do not type content into a button.
3. Execute only one action per iteration.
4. Strictly avoid repeating the same action if the webpage remains unchanged.
5. For complex tasks involving multiple questions or steps, select "stop" only at the very end.
6. Make sure all task requirements are satisfied before stopping.

WEB BROWSING GUIDELINES:

1. Do not interact with useless web elements such as Login, Sign-in, or donation buttons.
2. Visiting video websites is allowed, but the agent should not play videos.
3. Pay attention to filter and sorting functions, which can help solve conditions such as highest, cheapest, lowest, or earliest.
4. Pay attention to images and visual elements on the page.

EXAMPLE WORKFLOW:

{experience_memory}

Available actions:
{tools_section}

CRITICAL REQUIREMENTS:

1. Always use function calling. Never describe actions in plain text.
2. Provide clear reasoning in the reasoning parameter of each function call.
3. Be specific in descriptions to identify the correct elements.
4. Use proper JSON format for all function arguments.
5. Only execute one action at a time.
6. If an action fails, try a different approach or element.
7. For click and type actions, set a valid element_id corresponding to the numerical label of the target item.
8. For click and type actions, set valid coordinates in the format "<point>x1 y1</point>".
9. Use simple search terms when searching for information.
10. If the current page has no results, adjust the search term and try again.

CURRENT WEBPAGE OBSERVATION:

[Current webpage screenshot is provided as an image input.]

[Generated page description:]
{page_description}

FINAL PER-STEP TASK REMINDER:

****Current task:**** {current_task}

IMPORTANT REMINDERS:

- Please specify the number label of the item you want to interact with, in the description of the action.
- Do not repeat the same action multiple times. Try different approaches if something does not work.
- If your previous action is type, then you must click related pages or scroll pages to find the information you need.
- Pay attention to images, since many questions about shape, color, location, and visual content can be answered from screenshots.
- If the current search term yields no results, adjust the search term and try again.
- You must provide an answer within the remaining steps.
- Only output one action at a time. Do not output multiple actions at once.

What would you like to do next?

Remember: Your responses must be structured function calls that can be parsed directly by the system.

Mind2Web. Mind2Web is designed to evaluate whether an agent can follow natural language instructions to complete cross-page, multi-step web operation tasks on real-world websites. The original Mind2Web benchmark contains 2,350 open-ended tasks, covering 137 websites across 31 domains. In our GUI-agent project, we adapt the original test set through a unified format conversion and an execution-interface wrapper. Specifically, the version of Mind2Web used in our experiments mainly provides task-level information, including the task identifier, start URL, user intent, website category, and evaluation configuration, but does not include complete human action trajectories.

For the action space, we do not directly rely on the trajectory-level actions from the original dataset. Instead,

Table 11: Action Space of the GUI-Odyssey Dataset

Action	Action Format	Functionality
Click	<code>click [x, y]</code>	Tap a screen position
Long Press	<code>long_press [x, y]</code>	Long-press a screen position
Scroll	<code>scroll [x1, y1, x2, y2]</code>	Swipe from one position to another
Type	<code>type [text]</code>	Enter text
Complete	<code>complete</code>	Mark the task as completed
Impossible	<code>impossible</code>	Mark the task as impossible
Home	<code>home</code>	Return to the home screen
Back	<code>back</code>	Go back to the previous page
Recent	<code>recent</code>	Open recent apps

we integrate Mind2Web tasks into our interactive GUI environment. During evaluation, the agent is allowed to use the same action space as that used for MMInA, as described above. This unified action-space design enables Mind2Web and MMInA to be evaluated under the same GUI-agent execution framework, ensuring consistency in action representation, browser interaction, and task completion evaluation.

Table 10: Unified Action Space Used for Mind2Web and MMInA Evaluation

Action	Action Format	Functionality
Click	<code>click [element_id]</code>	Click a specified webpage element.
Type	<code>type [element_id] [text]</code>	Enter text into a specified input field.
Hover	<code>hover [element_id]</code>	Move the cursor over a webpage element.
Scroll	<code>scroll [up/down]</code>	Scroll the webpage upward or downward.
Press	<code>press [key_comb]</code>	Execute a keyboard shortcut or key command.
Go to URL	<code>goto [url]</code>	Navigate directly to a specified URL.
Go Back	<code>go_back</code>	Return to the previous page in browser history.
Go Forward	<code>go_forward</code>	Move forward in browser history.
New Tab	<code>new_tab</code>	Open a new browser tab.
Close Tab	<code>close_tab</code>	Close the current browser tab.
Page Focus	<code>page_focus [page_number]</code>	Switch focus to a specified browser page or tab.
Stop	<code>stop [answer]</code>	Terminate the task and return the final answer.

GUI-Odyssey. GUI-Odyssey is a dataset designed for mobile graphical user interface (GUI) agent tasks. It is primarily used to evaluate and train models on multi-step interaction capabilities in realistic mobile application environments. Each task in the dataset consists of a natural language instruction, a sequence of interface screenshots, and the corresponding action trajectory. Given the current screen state, an agent is expected to iteratively perform operations such as tapping, scrolling, text input, or system navigation in order to accomplish the user’s goal. In the version used in our work, the action space mainly covers nine common types of mobile GUI interactions: *Click*, *Long Press*, *Scroll*, *Type*, *Complete*, *Impossible*, *Home*, *Back*, and *Recent*, as detailed in Table 11.

Android-Control-v2. AndroidControl-v2 is an offline evaluation benchmark improved upon the original AndroidControl dataset. It pairs step-by-step task instructions, mobile interface screenshots, and human demonstration actions, and is designed to evaluate the step-level action prediction capability of GUI agents in Android environments. To improve data quality, AndroidControl-v2 filters samples by checking the consistency between annotated actions and oracle low-level instructions, resulting in 398 cleaned samples from the original 500 samples.

The dataset adopts a unified action space consisting of thirteen action types: Answer, Click, Select, LongPress, Write, KeyboardPress, Scroll, Swipe, Wait, NavigateHome, NavigateBack, OpenApp, and Terminate, as shown in

Table 12: Action Space of the AndroidControl-v2 Dataset

Action	Action Format	Functionality
Answer	answer [text]	Return the final answer to the user query
Click	click [target, x,y]	Tap or click a specific UI element
Select	select [target, option]	Select an option from a list or dropdown menu
LongPress	long_press [target, x,y]	Press and hold a UI element
Write	write [text, x,y]	Enter text into an input field
KeyboardPress	keyboard_press [key]	Press a specific keyboard key
Scroll	scroll [direction]	Scroll a view or container
Swipe	swipe [direction, x,y]	Perform a touchscreen swipe gesture
Wait	wait [seconds]	Pause execution for UI updates
NavigateHome	home	Navigate to the device home screen
NavigateBack	back	Press the system back button
OpenApp	open_app [app]	Launch a specified application
Terminate	terminate [message]	Signal the end of the current task

Table 12. These actions cover the fundamental operations required for mobile GUI interaction, including clicking, long pressing, text input, swiping, scrolling, system navigation, application launching, and task termination.

C Case Study

We provide qualitative case studies to illustrate how Mem-W uses latent memory during long-horizon GUI interaction. Figure 5 visualizes a web navigation example, where the agent must preserve task constraints and intermediate progress across multiple page transitions. Figure 6 presents a mobile interaction example, showing how the agent maintains an evolving record of the current episode while selecting executable GUI actions. To further reveal the role of experiential memory, Figure 7 shows the trajectories retrieved by Mem-W for the mobile case; these retrieved examples provide procedural evidence that complements the in-session working memory and guides the policy toward more reliable action choices.

D Limitation

While Mem-W demonstrates consistent improvements across diverse benchmarks, several aspects remain open for future investigation. First, the current trajectory-to-latent compressor is trained with a fixed latent-token budget, and adaptively allocating compression capacity based on trajectory complexity could further improve information retention. Also, although we evaluate on four benchmarks spanning web and mobile environments, the generalization of the learned latent memory to other GUI modalities such as desktop applications can also be explored, which we leave for future work. Finally, we view latent memory as a potentially model-agnostic interface rather than a mechanism tied to a particular backbone or tokenizer. Future work could explore more tokenizer-independent latent-token representations that can be transferred across models.

Case 01: Use Google Map to locate the nearest bookstore and then book a ride through Lyft to get there.



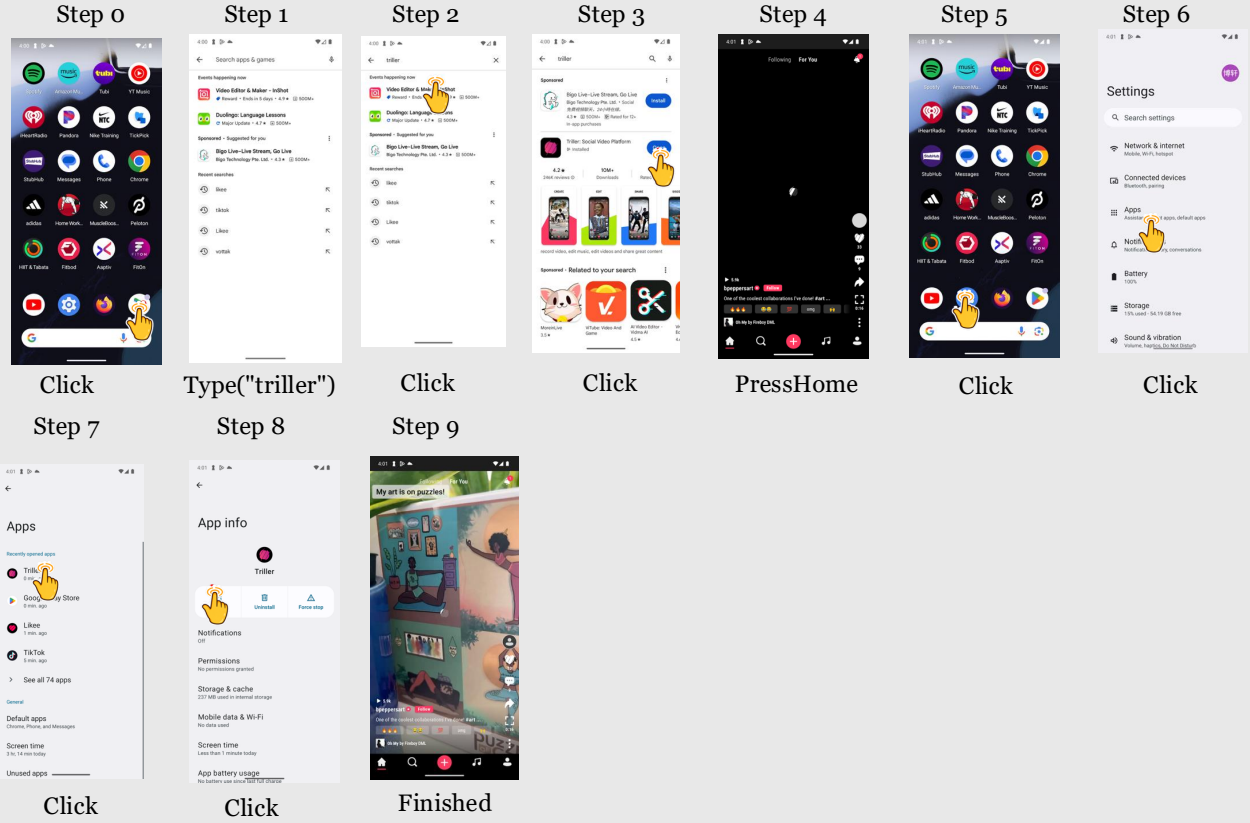
Figure 5: Case visualization I (website).

Task 02: First, install the Triller app using Google Play Store. Once installed, open the Triller app. Next, navigate to the Setting app and disable notifications for Triller. Finally, reopen the Triller app to watch a video.



Figure 6: Case visualization II (mobile).

Task02_experience01: First, navigate to the Google Play Store and install the Triller app. Once installed, open the Triller app. After this, go to the Setting app to disable notifications for Triller. Finally, reopen the Triller app and proceed to watch a video.



Task02_experience02: First, download and install the Triller video app from the Google Play Store. Once installed, open Triller. Next, navigate to the Setting app to disable notifications for Triller. Finally, reopen the Triller app to watch a video.

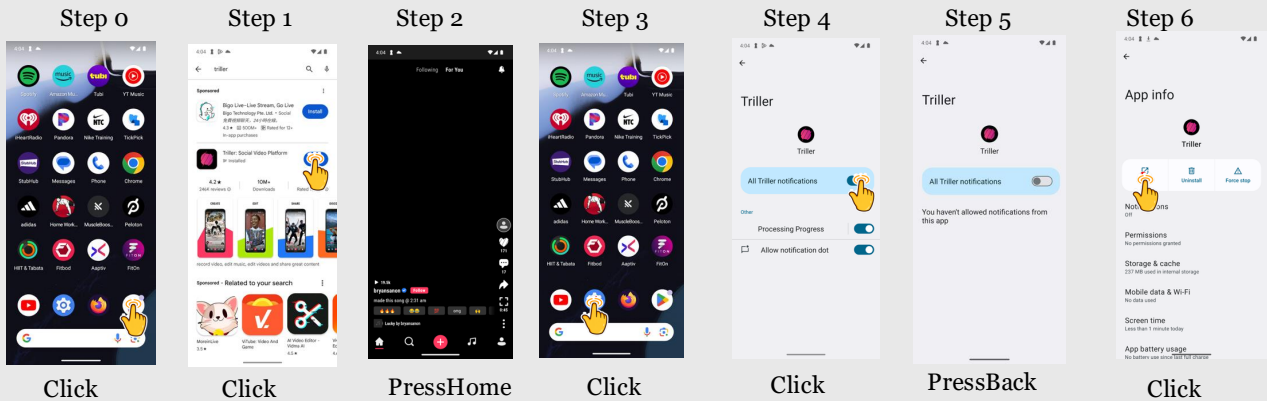


Figure 7: Case visualization II (the retrieved trajectories by Mem-W).